CPE 314 — Software Design & Software Design Laboratory

Week 8 — Project Proposal

Due end of Week 8 session | 10% of the Finals Term Grade | Scored out of 100

Group Name / No.:	PNP/1
Members:	Kirk Ryan Quisagan, Ken Tamidles, Kimberly Luistro, Yvonne Mae Tapan, Venice Nadela, Nesley Cate Librea
Repository URL:	https://github.com/Kimonooo/cpe314-69167-capstoneteam-Meal_plannerGroup1.git
Chosen Theme:	A B C D E F G Ⓗ I J K L M N O P Q R S T
API You Will Use:	TheMealDB :https://www.themealdb.com/api.php (shared test key "1"). Additional option under consideration: a nutrition API from RapidAPI's Food APIs collection (https://rapidapi.com/collection/food-apis)

1. One-Paragraph Pitch

Plate N Platter is a personal meal planner for individual users such as students and busy households. After registering and logging in, a user plans meals on a calendar where each meal is color-coded by Healthy Level and labeled Quick, Easy, or Hard to prepare. Users import real recipes from TheMealDB (including Filipino dishes) or create their own, including dishes built from other dishes, and the app adds up prep time, calories, and ingredients across them. A cooking queue, an undo stack, and a merged shopping list keep the user organized. Eating at least one Healthy meal a day builds a streak that earns reward badges, and a statistics page shows weekly charts and favorite dishes.

2. Entity-Relationship Sketch

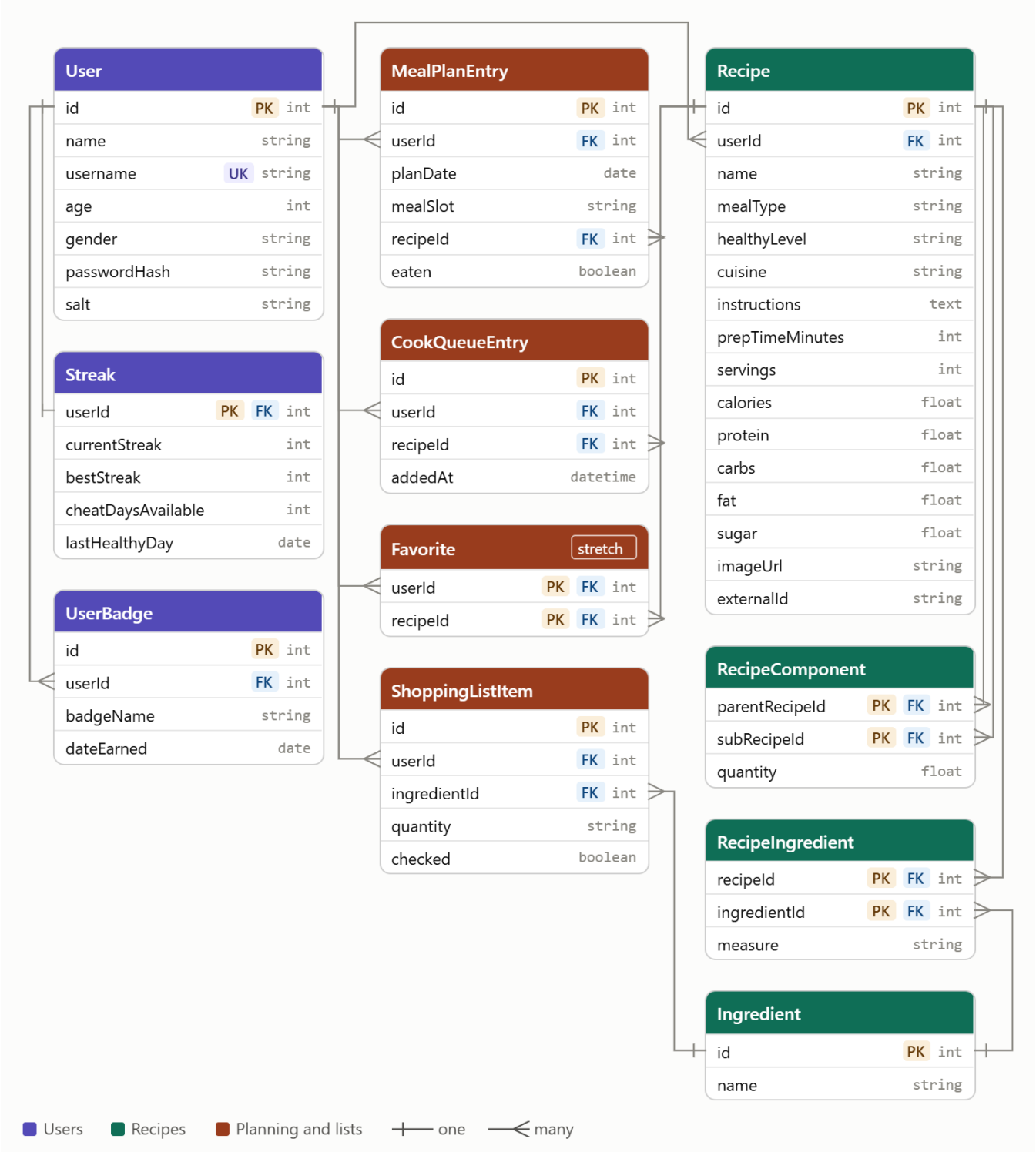


Figure 1. Preliminary Entity-Relationship Diagram

3. Feature Checklist

Check off which mandatory requirement each planned feature will satisfy:

Planned Feature	Requirement(s) It Satisfies
Register (name, username, age, gender, password, confirm password) and Login with salted password hashing and error messages	Database (JDBC PreparedStatement), Event-Driven GUI
Abstract Recipe with BreakfastRecipe, MainCourseRecipe, DessertRecipe, each with its own prep-time rule	Object-Oriented Design
Recursive methods that total prep time, calories, and ingredients across nested sub-recipes; total prep time then sets Quick / Easy / Hard (with protection against circular references)	Recursion
Cooking queue: "Add to Queue" on a recipe, first in, first out	Queue
Undo stack for adding, removing, and moving meals on the calendar	Stack
Shopping list that merges ingredients across planned meals (ingredient → total quantity)	Map
Weekly/monthly calendar grid with live date and time; meals colored by Healthy Level	Event-Driven GUI, Database
Today's Meals screen with Breakfast, Lunch, Dinner, and optional Snack, showing dish photos from the API	API Consumption, Event-Driven GUI
Meal Times screen: browse dishes, filter by Healthy Level and Low / Mid / High nutrition values, add to plan, and "Create your own meal" form	Event-Driven GUI (3+ controls), Database
Import recipes from TheMealDB by name, category, or ingredient, parsed from JSON into Recipe objects	API Consumption
API search and large CSV imports (recipe box and meal-plan history) run on a background Task with a loading indicator	Concurrency
Weekly statistics: meals by Healthy Level, most-planned category and cuisine, most-eaten dishes, average prep time, average daily calories	Data Mining
Statistics bar chart of meals per Healthy Level (weekly summary)	Data Visualization
Daily streak: eating at least one Healthy-level meal each day (marked eaten) continues the streak; milestones award badges (Full Week Planner, Recipe Collector, Perfect Week, Healthy Meal Planned) and cheat-day rewards	Data Mining (consecutive-day analysis of meal-plan records), Event-Driven GUI
6+ JUnit 5 tests: password hashing, recursion (plain recipe, one sub-recipe, 3-level nest, empty recipe), queue order, undo stack, shopping-list merge, streak counting	Automated Testing
Filipino Dishes category: filter TheMealDB by cuisine (Filipino), supplemented by our own Filipino recipe CSV if the API has few	API Consumption

Optional cloud sync of the SQLite tables to Cloudflare D1 (needs instructor approval)	Database (extension)
Look up calories and nutrients for imported dishes from a nutrition API (additional option) and cache results in SQLite; used to help set Healthy Level	API Consumption, Database

4. Task Split

Member	Primary Responsibility (this milestone)
Kimberly Luistro	**Team Leader**: • Builds the Recipe classes (BreakfastRecipe, MainCourseRecipe, DessertRecipe) and MealPlanEntry. • Implements the cooking queue and undo stack. • Sets milestones, reviews pull requests, and handles submissions.
Nesley Cate Librea	**UI:** • Builds the login/register screens, Home layout with side menu, and the Recipe Details and Shopping List screens. • Writes the shared maroon-and-cream stylesheet.
Yvonne Mae Tapan	**UI:** • Builds the calendar grid, Today's Meals, Meal Times (filters, dish cards, "Create your own meal"), and the Streak and Statistics screens with the bar chart.
Venice Nadela	**QA**: • Writes the 6+ JUnit 5 tests, keeps a manual test checklist per screen, tests login errors, circular references, CSV imports, and failed • API calls, and tracks bugs in GitHub Issues.
Ken Tamidles	**Database Admin:** • Creates the SQLite tables and JDBC data-access classes with PreparedStatement. Stores salted password hashes, caches API results, and keeps API keys out of the repository.
Kirk Quisagan	**Developer**: • Writes the TheMealDB client and JSON parsing, the recursive prep-time, calorie, and ingredient totals with circular-reference protection, and the background Task for API and CSV imports. Also writes the shopping-list merge, streak logic, and weekly statistics.

5. Public API You Plan to Use

TheMealDB (https://www.themealdb.com/api.php, endpoints under https://www.themealdb.com/api/json/v1/1/) provides real recipe data, including each meal's name, category, cuisine, step-by-step instructions, photo, and ingredient list with measures, which our app searches, imports, and saves into its local database.

6. Risks / Open Questions

● TheMealDB as our API has no calories, nutrition, or prep-time data. We are considering a nutrition API from RapidAPI's Food APIs collection (Spoonacular, Edamam, or CalorieNinjas) to supply calories and nutrients for imported dishes, with user entry as the fallback. Is a second API acceptable?

● TheMealDB categories (Beef, Chicken, etc.) differ from our Breakfast / MainCourse / Dessert, so we need a mapping rule. We also need to confirm how many Filipino dishes the API actually offers; if few, we will add our own.

● Sub-recipes are not provided by the API. Users will link them in the app. Hence, we must prevent circular references .

● Are Quick / Easy / Hard thresholds based on the recursive total prep time acceptable? The same recursion also totals calories and ingredients.

● Database: the spec requires SQLite tables accessed via JDBC with PreparedStatement. Cloudflare D1 is SQLite-compatible but is reached over the internet (a community JDBC driver exists, or Cloudflare's REST API / a Worker). We propose local SQLite via JDBC as the main database and D1 as an optional cloud sync. Do we need approval to use D1, and is an API token inside a desktop app acceptable?